Sistemas Distribuídos — Ficha REP01

Comunicação Indireta com Multicast e Publish/Subscribe

Pedro (Popeye1981-Viseu)

14 de outubro de 2026

Conteúdo

1 Introdução **2**
1.1 Multicast IP . . . 2
1.2 Publish/Subscribe com PubSub do IPFS . . . 2
1.3 Tema: replicação de um ficheiro de registos . . . 3

2 Parte A — Replicação com Multicast **3**
2.1 Classes . . . 3
2.1.1 `SensorRecord` . . . 3
2.1.2 `RecordFile` . . . 4
2.1.3 `MulticastLeader` . . . 5
2.1.4 `MulticastReplica` . . . 6
2.2 Testes e resultados . . . 7

3 Parte B — Replicação com PubSub do IPFS **7**
3.1 Reutilização da Parte A . . . 7
3.2 Classes . . . 7
3.2.1 `IpfsPubSub` . . . 7
3.2.2 `PubSubLeader` . . . 9
3.2.3 `PubSubReplica` . . . 9
3.3 IPFS — Kubo e daemons . . . 11
3.3.1 Instalação e primeiro nó . . . 11
3.3.2 Segundo nó (porto 5002) . . . 11
3.3.3 Ligação entre nós . . . 11
3.3.4 Comandos úteis . . . 12
3.3.5 Uma ilha ou uma malha . . . 12
3.4 Testes e resultados . . . 12

4 Verificação — Ponto 14 (Falhas) **13**

5 Verificação — Ponto 15 (Construções) **14**

6 Comparação e Reflexão **15**
6.1 Ponto 16 — Tabela comparativa . . . 15
6.2 Ponto 17 — Reflexão . . . 16

7 Conclusão **17**

1 Introdução

Na ficha TCP01, a comunicação era *direta*: o cliente tinha de conhecer o endereço e o porto do servidor, e os dois processos tinham de estar em execução ao mesmo tempo. Na **comunicação indireta**, o emissor e o recetor comunicam através de um *intermediário* e não têm ligação direta entre si. Quem envia dirige a mensagem ao intermediário — um *grupo*, um *tópico* — e não a um processo concreto; é o intermediário que a faz chegar a quem estiver interessado.

A comunicação indireta tem duas propriedades que a distinguem da direta:

- **Desacoplamento no espaço** — o emissor não sabe *quem* são os recetores, nem *quantos* são. Podem entrar e sair recetores sem que o emissor mude uma linha de código.
- **Desacoplamento no tempo** — emissor e recetor *não precisam de existir ao mesmo tempo*: uma mensagem enviada agora pode ser entregue a um recetor que só arranca mais tarde.

É importante sublinhar que **a comunicação indireta não é o mesmo que comunicação fiável**. O intermediário decide *a quem* a mensagem chega, mas não garante, por si só, *que* chega, *quantas vezes* chega, nem *por que ordem*.

1.1 Multicast IP

Na **comunicação em grupo**, uma mensagem é enviada a um *grupo de processos* e entregue a todos os seus membros. O **multicast IP** é a forma mais simples de o fazer: o emissor envia **um único datagrama** para um *endereço de grupo* (IPv4: `224.0.0.0` a `239.255.255.255`) e a *rede* entrega uma cópia a cada máquina que se *juntou* a esse grupo.

O grupo é *aberto* — qualquer processo pode enviar-lhe mensagens, mesmo sem ser membro — e *dinâmico*: os membros entram (`joinGroup`) e saem (`leaveGroup`) quando querem, e o emissor nunca chega a saber quantos são. O custo de envio é o mesmo para um ou para cem membros.

O multicast IP assenta em **UDP** e herda as suas garantias — ou a falta delas: não há ligação, confirmação, retransmissão nem ordem garantida. Um membro que não esteja à escuta no momento do envio *perde* a mensagem, e ninguém é avisado. Além disso, a maior parte dos *routers* da Internet **não encaminha** multicast: na prática, funciona dentro de uma rede local.

1.2 Publish/Subscribe com PubSub do IPFS

Num sistema *publish/subscribe*, os emissores *publicam* mensagens (também chamadas *eventos*) e os recetores declaram o seu interesse através de *subscrições*. O sistema entrega cada mensagem publicada a todos os subscritores interessados. Nesta ficha usa-se a forma mais comum, baseada em *tópicos*: cada mensagem é publicada num tópico com nome, e quem subscreve esse tópico recebe todas as mensagens nele publicadas.

A diferença em relação ao multicast está no intermediário. No multicast, quem entrega é a *rede*, com base num endereço IP. No pubsub, quem entrega é um *serviço* que sabe quem subscreveu o quê — seja um servidor central (*broker*), seja uma rede de nós que cooperam entre si.

O **IPFS** é uma rede *peer-to-peer*. Cada máquina corre um *nó* — nesta ficha, o daemon **Kubo** (`ipfs`) — ligado a outros nós. O PubSub do IPFS usa o protocolo *gossipsub* da biblioteca libp2p: cada nó mantém ligações a alguns vizinhos interessados no mesmo tópico e *reencaminha* as mensagens que recebe, até que cheguem a todos os subscritores. Não há *broker*: o intermediário é a própria rede de nós, e as mensagens podem atravessar a Internet, porque viajam sobre as ligações normais entre eles.

Tal como o multicast, o PubSub do IPFS é *best-effort*: as mensagens *não ficam guardadas* em lado nenhum, um subscritor que não esteja ligado no momento da publicação perde-as, e a ordem

de chegada não é garantida. Por outro lado, o gossipsub *elimina duplicados* e cada mensagem identifica o nó que a publicou (campo `from`).

1.3 Tema: replicação de um ficheiro de registos

Para pôr os dois mecanismos à prova, usa-se um problema em que a comunicação um-para-muitos surge naturalmente: manter várias cópias de um ficheiro de leituras de sensores. Um processo *líder* é o único que aceita novos registos; sempre que regista um, propaga-o às *réplicas*, que o acrescentam ao seu próprio ficheiro. É aqui que o desacoplamento no espaço se torna útil: o líder não precisa de uma lista de réplicas, e uma réplica nova junta-se ao grupo ou ao tópico sem que o líder mude.

Cada registo é uma linha de texto com quatro campos separados por `;`, usada tanto no ficheiro como na mensagem:

```
seq;sensor;temperatura;instante
17;S01;21.5;2026-10-01T10:15:03
```

Ao contrário da ficha TCP01, **não se usa serialização nativa do Java**: o PubSub do IPFS é acessível a programas escritos em qualquer linguagem, e uma linha de texto é interpretável por todos. Por isso o `SensorRecord` não implementa `Serializable`.

Como nenhum dos mecanismos garante entrega nem ordem, é a aplicação que tem de lidar com isso, através do **número de sequência** (`seq`), atribuído *só pelo líder*, de forma crescente e sem saltos:

- recebeu `seq = último + 1` → registo esperado: **escreve-o** no ficheiro e, a seguir, escreve os registos retidos que passaram a estar em sequência;
- recebeu `seq <= último` → **duplicado**: ignora-o;
- recebeu `seq > último + 1` → **fora de ordem**: não o escreve — guarda-o numa *estrutura em memória* até chegarem os que faltam.

A regra é estrita: o ficheiro de uma réplica só cresce pela ordem do líder, sem saltos. **Nesta ficha não existe recuperação**: nenhuma réplica pede ao líder os registos em falta. Se uma mensagem se perder de vez, a réplica fica retida nesse ponto.

2 Parte A — Replicação com Multicast

2.1 Classes

2.1.1 `SensorRecord`

Responsabilidade: representa uma leitura de sensor. Contém quatro campos imutáveis (`final`): `seq` (`long`), `sensorId` (`String`), `temperature` (`double`) e `timestamp` (`String`). Define o **contrato textual de comunicação** entre líder e réplica.

```
1         public String toLine() {
2             return seq + ";" + sensorId + ";" + temperature + ";" + timestamp;
3         }
4
5         public static SensorRecord fromLine(String line) {
6             String[] f = line.trim().split(";");
7             if (f.length != 4) {
8                 throw new IllegalArgumentException("Registo mal formado: " +
                      line);
9             }
```

```
10              return new SensorRecord(Long.parseLong(f[0]), f[1],
11              Double.parseDouble(f[2]), f[3]);
12          }
```

Listing 1: Serialização e desserialização em texto

Fábrica `now(...)`:

```
1           public static SensorRecord now(long seq, String sensorId, double
                temperature) {
2               return new SensorRecord(seq, sensorId, temperature,
3               LocalDateTime.now().withNano(0).toString());
4           }
```

Listing 2: Fabrica usada apenas pelo lider

O `withNano(0)` trunca os nanossegundos do timestamp. O `toString()` do `LocalDateTime` produz então o formato ISO-8601, legível por qualquer linguagem.

Porque é `static`: os métodos `now(...)` e `fromLine(...)` são *fábricas* — criam objetos novos a partir de zero, sem depender de nenhuma instância existente. Se fossem não-estáticos, para os chamar terias de ter *já* um `SensorRecord` criado, o que é absurdo. Nomes descritivos (`now`, `fromLine`) tornam o código mais legível do que construtores sobrecarregados.

Porque não implementa `Serializable`: o transporte é feito em *texto* (`toLine`/`fromLine`), não em serialização binária do Java. Isto permite interoperabilidade com qualquer linguagem.

O que não garante:

- Não valida o formato do `timestamp` (aceita `"hoje"` — só o líder gera timestamps corretos).
- Não valida o intervalo da temperatura (aceita `-999` ou `999999`).
- Não valida se o `sensorId` é plausível.
- Não valida se o `seq` é positivo.

O recetor deteta corrupção indiretamente via `f.length != 4` — um `;` dentro do `sensorId` seria apanhado, mas só porque quebra a contagem de campos.

2.1.2 `RecordFile`

Responsabilidade: persistência local em ficheiro de texto.

```
1           public RecordFile(String fileName) {
2               this.path = Paths.get(fileName);    // apenas guarda referencia; nao
                    abre
3           }
4
5           public synchronized void append(SensorRecord r) throws IOException {
6               Files.writeString(path, r.toLine() + System.lineSeparator(),
7               StandardCharsets.UTF_8,
8               StandardOpenOption.CREATE, StandardOpenOption.APPEND);
9           }
10
11          public synchronized long lastSeq() throws IOException {
12              if (!Files.exists(path)) return 0;
13              long last = 0;
14              for (String line : Files.readAllLines(path, StandardCharsets.UTF_8))
                    {
15                  if (!line.isBlank()) {
16                      last = SensorRecord.fromLine(line).getSeq();
17                  }
18              }
19              return last;
```

```
20          }
```

Listing 3: Escrita e leitura do ficheiro

O construtor `new RecordFile("lider.txt")` **não abre o ficheiro** — só guarda um `Path`. O ficheiro só é tocado quando `append` ou `lastSeq` são chamados.

Bloqueios:

- `synchronized` — lock entre threads da mesma JVM.
- `Files.writeString`/`readAllLines` — I/O bloqueante à espera do SO.

O que não garante:

- Não valida os dados (confia no `toLine()`).
- Não garante ordem — devolve o `seq` da *última linha*, não o máximo.
- Não garante durabilidade (*fsync*) — se a máquina crashar, o último registo pode perder-se.
- Não é atómico entre processos — o `synchronized` só funciona dentro da mesma JVM.
- Não deteta corrupção externa — se o ficheiro for editado manualmente, `lastSeq` rebenta no parsing.

2.1.3 `MulticastLeader`

Responsabilidade: gerir o ciclo interativo do líder. Lê comandos da CLI, atribui números de sequência crescentes, grava em `lider.txt` e propaga para o grupo `230.0.0.1:6789` com TTL=1. Suporta os comandos `<sensor> <temperatura>`, `rajada <n>` e `sair`.

```
1           MulticastSocket socket = new MulticastSocket();
2           InetAddress group = InetAddress.getByName(GROUP);    // "230.0.0.1"
3           socket.setTimeToLive(1);                             // nao sai da LAN
```

Listing 4: Setup do socket multicast

```
1           SensorRecord r = SensorRecord.now(++seq, p[0], temp);
2           file.append(r);                                      // 1) escreve
                localmente
3           byte[] m = r.toLine().getBytes(StandardCharsets.UTF_8);
4           socket.send(new DatagramPacket(m, m.length, group, PORT)); // 2) propaga
5           System.out.println("Registado e enviado: " + r.toLine());
```

Listing 5: Ordem: append local antes de propagar

Justificação da ordem `append` → `send`: o líder é a fonte autoritária dos números de sequência. Se o processo falhar entre as duas operações, o líder fica **à frente** das réplicas (tem o registo, elas não). A ordem inversa deixaria as réplicas com um registo que o líder não reconhece — pior, porque a réplica ficaria com um `seq` que o líder depois regeneraria. É preferível que o líder saiba mais do que as réplicas do que o contrário.

Bloqueios:

- `sc.nextLine()` — espera input do utilizador.
- `file.append(r)` — espera confirmação do disco.
- `socket.send(...)` — espera o buffer do SO (microssegundos; não espera por réplica).

O que não garante: número de réplicas, entrega, ordem, ausência de duplicados, confirmação de receção. O líder é **cego ao que se passa do outro lado** — o `send` retorna OK mesmo que ninguém esteja à escuta.

2.1.4 `MulticastReplica`

Responsabilidade: juntar-se ao grupo `230.0.0.1`, receber datagramas, desserializá-los e aplicar a **lógica de sequência**.

```
        if (seq <= last || pending.containsKey(seq)) {
            System.out.println("DUPLICADO: " + seq);
        } else if (seq == last + 1) {
            file.append(r);
            last = seq;
            System.out.println("Aplicado: " + r.toLine());
            // drenar pending em cascata
            while (pending.containsKey(last + 1)) {
                SensorRecord next = pending.remove(last + 1);
                file.append(next);
                last = next.getSeq();
                System.out.println("Aplicado (retido): " + next.toLine());
            }
        } else {
            pending.put(seq, r);
            System.out.println("EM ESPERA: " + seq
            + " (falta o registo " + (last + 1) + ")");
        }
```

Listing 6: Lógica de sequencia e retencao

Porque `TreeMap`: mantém as chaves ordenadas. Um `HashMap` seria mais rápido a inserir (O(1) vs O(log n)), mas o `TreeMap` permite iterar em ordem crescente sem ordenação adicional, e oferece `firstKey()`/`lastKey()` para inspeção. Como o número de retidos é pequeno, a diferença de performance é irrelevante — a clareza conceptual compensa.

Parsing isolado:

```
        SensorRecord r;
        try {
            r = SensorRecord.fromLine(line);
        } catch (IllegalArgumentException e) {
            System.out.println("REJEITADO: " + line);
            continue;
        }
```

Listing 7: Rejeicao de datagramas malformados sem terminar a replica

O `catch` apanha `IllegalArgumentException`, que é a superclasse comum de `NumberFormatException`. Se apanhásse apenas `RuntimeException`, esconderia bugs de programação (`NullPointerException`, etc.).

Heurística de fim de rajada:

```
        socket.setSoTimeout(1000);
        ...
        try {
            socket.receive(p);
            line = new String(p.getData(), 0, p.getLength(), StandardCharsets.
                UTF_8);
        } catch (SocketTimeoutException e) {
            if (rajadaAtiva) {
                System.out.println("=== FIM DA RAJADA ===");
                System.out.println("t0=" + rajadaT0 + " | t1=" + rajadaT1
                + " | janela de rececao=" + (rajadaT1 - rajadaT0) + " ms");
                // ...
            }
            continue;
        }
```

Listing 8: Timeout de 1s para detetar o fim da rajada

Diferença bloqueio físico vs lógico: o timeout reinicia apenas os *contadores da rajada* — não toca no `last` nem no `pending`. O ficheiro continua *congelado* se um `seq` se perdeu.

O que não garante: recuperação de registos perdidos. Se um `seq` intermédio nunca chegar, a réplica fica congelada e o `pending` cresce sem limite. O `seq` *deteta*, mas não *recupera*.

2.2 Testes e resultados

Teste 4 (5 registos, 3 réplicas): ficheiros do líder e das réplicas ficaram todos iguais. Multicast funcional numa rede local.

Teste 5 (parar réplica 2, 3 registos, reiniciar): a réplica 2 ficou sem os 3 registos. **Nenhum programa detetou o problema** — divergência silenciosa.

Teste de retenção (9, 6, 7, 8): a réplica com `last=5` imprimiu `EM ESPERA: 9 (falta o registo 6)`, e drenou os retidos após a chegada dos `6, 7, 8`. Ficheiro final ordenado, sem saltos.

Rajada 1000: com uma réplica previamente bloqueada, todas as 1000 mensagens foram recebidas mas `Escritos=0` — o `pending` acumulou (2533, 2525, 2520). Demonstra o bloqueio lógico permanente.

3 Parte B — Replicação com PubSub do IPFS

3.1 Reutilização da Parte A

As classes `SensorRecord` e `RecordFile` foram **reutilizadas tal-e-qual** da Parte A. A serialização em texto (`toLine`/`fromLine`) e a persistência local são *independentes do mecanismo de transporte*. Esta é a vantagem de separar *modelo de dados* e *propagação*: mudar de multicast para PubSub não exige alterar o formato nem o ficheiro.

3.2 Classes

3.2.1 `IpfsPubSub`

Responsabilidade: abstração sobre a API HTTP do Kubo. Esconde multibase, multipart e JSON streaming. Líder e réplica só veem `publish(topic, message)` e `subscribe(topic, callback)`. É o equivalente PubSub do `MulticastSocket` da Parte A.

```
        static String toMultibase(String s) {
            return "u" + Base64.getUrlEncoder().withoutPadding()
            .encodeToString(s.getBytes(StandardCharsets.UTF_8));
        }

        static String fromMultibase(String s) {
            if (!s.startsWith("u"))
            throw new IllegalArgumentException("Codificacao multibase nao
                suportada");
            return new String(Base64.getUrlDecoder().decode(s.substring(1)),
            StandardCharsets.UTF_8);
        }
```

Listing 9: Codificacao multibase exigida pela API do Kubo

O `toMultibase` produz, por exemplo, `rep01-temperaturas` → `ucmVwMDEtdGVtcGVyYXR1cmFz`. A API do Kubo exige este formato para o `arg` (tópico) e para o campo `data` das mensagens.

```
        public void publish(String topic, String message)
        throws IOException, InterruptedException {
            String boundary = "rep01-" + System.nanoTime();
            String body = "--" + boundary + "\r\n"
```

```
            + "Content-Disposition: form-data; name=\"file\"; filename=\"msg\"\r\n"
            + "Content-Type: application/octet-stream\r\n\r\n"
            + message + "\r\n"
            + "--" + boundary + "--\r\n";
            HttpRequest req = HttpRequest.newBuilder(
            URI.create(api + "/pubsub/pub?arg=" + toMultibase(topic)))
            .header("Content-Type", "multipart/form-data; boundary=" + boundary)
            .POST(HttpRequest.BodyPublishers.ofString(body, StandardCharsets.UTF_8))
            .build();
            HttpResponse<String> resp = http.send(req, HttpResponse.BodyHandlers.ofString());
            if (resp.statusCode() != 200)
            throw new IOException("pub falhou (" + resp.statusCode() + ")");
        }
```

Listing 10: Publicar — bloqueia ate o daemon responder

O `boundary` é um separador aleatório (não é o tópico!). O tópico vai no URL. O corpo está em `multipart/form-data` porque o Kubo o exige — a mensagem é enviada como se fosse um ficheiro.

```
        public void subscribe(String topic, Consumer<String> onMessage)
        throws IOException, InterruptedException {
            HttpRequest req = HttpRequest.newBuilder(
            URI.create(api + "/pubsub/sub?arg=" + toMultibase(topic)))
            .POST(HttpRequest.BodyPublishers.noBody())
            .build();
            HttpResponse<Stream<String>> resp =
            http.send(req, HttpResponse.BodyHandlers.ofLines());
            if (resp.statusCode() != 200)
            throw new IOException("sub falhou (" + resp.statusCode() + ")");
            resp.body()
            .filter(json -> json.contains("\"data\""))
            .forEach(json -> onMessage.accept(
            fromMultibase(extractField(json, "data"))));
        }
```

Listing 11: Subscrever — bloqueia PARA SEMPRE no forEach

Assimetria crucial: `publish` bloqueia por milissegundos (finito); `subscribe` bloqueia **para sempre** no `forEach` — a thread só sai se a ligação HTTP cair. Não é assíncrono (`send`, não `sendAsync`).

```
        static String extractField(String json, String field) {
            String key = "\"" + field + "\":\"";
            int start = json.indexOf(key);
            if (start < 0)
            throw new IllegalArgumentException("Campo '" + field + "' ausente: "
                + json);
            start += key.length();
            return json.substring(start, json.indexOf('"', start));
        }
```

Listing 12: Extracao minimalista de campo JSON (sem parser real)

É uma busca de strings, não um parser real. Não interpreta escapes nem valida JSON. Justificação: a ficha proíbe bibliotecas externas e o `java.util` não tem parser JSON.

O que não garante: entrega, ordem, ausência de duplicados entre nós, daemon vivo, PubSub ativado, existência de subscritores. Acrescenta conveniência, não fiabilidade.

3.2.2 PubSubLeader

Responsabilidade: idêntica ao `MulticastLeader`, mas o transporte é `pubsub.publish(...)` em vez de `socket.send(...)`. Ficheiro continua a ser `lider.txt`. Aceita um argumento opcional com o URL da API.

```
        SensorRecord r = SensorRecord.now(++seq, p[0], temp);
        file.append(r);
        pubsub.publish(TOPIC, r.toLine());    // <-- em vez de socket.send(...)
        System.out.println("Registado e enviado: " + r.toLine());
```

Listing 13: Unica mudanca face ao MulticastLeader

Bloqueios:

- `new IpfsPubSub(api)` — **não bloqueia** (só guarda referências; o `HttpClient` é criado localmente sem ligação TCP).
- `sc.nextLine()` — input do utilizador.
- `file.append(r)` — disco.
- `pubsub.publish(...)` — resposta HTTP do daemon local.

O que não garante: daemon vivo, PubSub ativado, propagação pelo *gossip*, entrega efetiva, ordem. Acrescenta uma dependência (o daemon) sem acrescentar garantias.

3.2.3 PubSubReplica

Responsabilidade: subscrever o tópico, processar cada mensagem com a mesma lógica de sequência da `MulticastReplica`, e medir a rajada com uma heurística de silêncio de 1 segundo.

Três threads:

- **main** — chama `subscribe`, fica bloqueada no `forEach`.
- **subscribe** (criada internamente pelo `HttpClient`) — executa a lambda `line -> {...}` por cada mensagem.
- **fim-rajada** (criada pelo `ScheduledExecutorService`) — executa `imprimirResumoRajada` 1s após o último SIM.

Sincronização: todo o estado partilhado (`file`, `pending`, contadores, tokens) está protegido por `synchronized (LOCK)` — um único objeto lock acessível a ambas as threads. Sem o `LOCK`, a thread `fim-rajada` podia ler contadores a meio de uma atualização, produzindo um resumo inconsistente.

```
        private static final Object LOCK = new Object();
        private static RecordFile file;
        private static final TreeMap<Long, SensorRecord> pending = new TreeMap
            <>();

        private static final ScheduledExecutorService SCHED =
        Executors.newSingleThreadScheduledExecutor(r -> {
            Thread t = new Thread(r, "fim-rajada");
            t.setDaemon(true);
            return t;
        });
        private static ScheduledFuture<?> tarefaFim = null;

        private static long versaoTarefa = 0;
        private static long geracao = 0;
```

```
15          private static boolean rajadaAtiva = false;
16          private static long rajadaT0 = 0, rajadaT1 = 0;
17          private static int rajadaRecebidos = 0, rajadaEscritos = 0;
```

Listing 14: Setup do agendador e dos dois tokens

```
1           private static void rearmarFimRajada() {
2               if (tarefaFim != null) {
3                   tarefaFim.cancel(false);            // nao interrompe se ja
                        comecou
4               }
5               versaoTarefa++;                         // muda em CADA rearm
6               final long versaoAlvo = versaoTarefa;
7               final long geracaoAlvo = geracao;       // so para diagnostico
8               tarefaFim = SCHED.schedule(
9               () -> imprimirResumoRajada(versaoAlvo, geracaoAlvo),
10              1, TimeUnit.SECONDS);
11          }
12
13          private static void imprimirResumoRajada(long versaoAlvo, long
                geracaoAlvo) {
14              synchronized (LOCK) {
15                  if (versaoAlvo != versaoTarefa) return;   // tarefa obsoleta
16                  if (!rajadaAtiva) return;
17                  System.out.println("=== FIM DA RAJADA === [geracao=" +
                        geracaoAlvo + "]");
18                  System.out.println("t0=" + rajadaT0 + " | t1=" + rajadaT1
19                  + " | janela de rececao=" + (rajadaT1 - rajadaT0) + " ms");
20                  System.out.println("Recebidos=" + rajadaRecebidos
21                  + " | Escritos=" + rajadaEscritos
22                  + " | Retidos=" + pending.size());
23                  rajadaAtiva = false;
24                  rajadaT0 = rajadaT1 = 0;
25                  rajadaRecebidos = rajadaEscritos = 0;
26                  tarefaFim = null;
27              }
28          }
```

Listing 15: Dois tokens: versaoTarefa invalida; geracao distingue

Porque existem os dois tokens:

- `versaoTarefa` — incrementa *a cada* rearm (a cada SIM). **Invalida tarefas obsoletas**: se uma tarefa antiga já começou a executar (ou está à espera do `LOCK`), vê `versaoAlvo != versaoTarefa` e sai sem fazer nada.
- `geracao` — incrementa apenas quando começa uma nova rajada (`rajadaAtiva` passa de `false` para `true`). Distingue janelas de rajada e serve para diagnóstico.

Sem o `versaoTarefa`, uma tarefa antiga que tenha começado a correr dentro da mesma rajada fecharia a rajada prematuramente. Sem o `geracao`, não saberíamos em que rajada estamos.

Padrão *debounce*: cada SIM cancela a tarefa anterior e agenda uma nova para daqui a 1s. Só a última (que sobrevive ao silêncio) fecha a rajada. Sem o rearm, o alarme dispararia a meio da rajada. Este é o mesmo padrão da barra de pesquisa do Google (espera 300ms após a última tecla).

Bloqueios:

- `pubsub.subscribe(...)` — **para sempre** no `forEach`.
- `synchronized (LOCK)` — espera pela outra thread.
- `file.append(r)` — I/O de disco.

- `SCHED.schedule(...)` — **não bloqueia** (agenda e segue).

Diferença bloqueio físico vs lógico: o timeout reinicia apenas os contadores da rajada (bloqueio físico da thread `fim-rajada`); o ficheiro continua *congelado* em `last` e o `pending` cresce (bloqueio lógico).

O que não garante: recuperação de registos perdidos no *gossip*, saída do estado de bloqueio, sincronização com o líder, precisão absoluta do resumo (heurística de 1s é um risco assumido).

3.3 IPFS — Kubo e daemons

3.3.1 Instalação e primeiro nó

- `wget https://github.com/ipfs/kubo/releases/...` — descarregar o binário correto (`linux-amd64`; `darwin` é macOS).
- `sudo bash install.sh` — instalar.
- `ipfs init` — cria `~/.ipfs` com chave privada e PeerID.
- `ipfs config -json Pubsub.Enabled true` — ativa PubSub (antes de arrancar o daemon).
- `ipfs daemon` — expõe três portas:
  - **4001** (Swarm P2P) — comunicação entre nós IPFS.
  - **5001** (API RPC HTTP) — é aqui que o código Java fala.
  - 8080 (Gateway HTTP) — usado pelo browser; não usamos.

Nota: o daemon fica em *foreground* — bloqueia o terminal. Tem de estar *sempre* a correr para o código Java funcionar.

3.3.2 Segundo nó (porto 5002)

```
export IPFS_PATH=~/.ipfs-no2
ipfs init
ipfs config --json Pubsub.Enabled true
ipfs config Addresses.API /ip4/127.0.0.1/tcp/5002
ipfs config Addresses.Gateway /ip4/127.0.0.1/tcp/8081
ipfs config --json Addresses.Swarm '["/ip4/0.0.0.0/tcp/4002","/ip4
    /0.0.0.0/udp/4002/quic-v1"]'
ipfs daemon
```

Listing 16: Comandos do segundo no (bash)

Cada nó tem o seu repositório (`~/.ipfs` e `~/.ipfs-no2`) e as suas portas. `IPFS_PATH` só vive na sessão do terminal atual.

3.3.3 Ligação entre nós

```
# Obter o PeerID do no 2
export IPFS_PATH=~/.ipfs-no2
ipfs id -f '<id>'          # -> 12
    D3KooWPgAkTCKFMtTYpkHkATsemiK6CiZF2mxTdvg68gVrRuvb

# Do no 1, ligar ao no 2
ipfs swarm connect /ip4/127.0.0.1/tcp/4002/p2p/12D3KooWPgAkT...
# -> connect 12D3KooW... success

```

```
9         # Confirmar
10        ipfs swarm peers
11        ipfs pubsub peers rep01-temperaturas
```

Listing 17: Ligar os dois nos via swarm connect

A ligação `swarm connect` **não é persistente** — se reiniciar qualquer daemon, é preciso voltar a ligar.

3.3.4 Comandos úteis

- `ipfs id` / `ipfs version` — estado do nó.
- `ipfs config Pubsub.Enabled` — verificar PubSub.
- `ipfs swarm peers` / `ipfs swarm connect <multiaddr>` — gerir ligações P2P.
- `ipfs pubsub ls` / `ipfs pubsub peers <topico>` — inspecionar.
- `ipfs pubsub sub <topico>` — subscrever na CLI.
- `echo "msg ipfs pubsub pub <topico>` — publicar na CLI.
- `echo $IPFS_PATH` — ver em que repositório estás.

3.3.5 Uma ilha ou uma malha

Com 1 nó: todos os clientes Java ligam-se ao mesmo daemon (5001). A entrega é *local* — cópia de memória dentro do mesmo processo, sem travessia de rede. **Sem perdas** (1000/1000 em todas as execuções).

Com 2 nós: o líder publica no daemon 1. Este entrega localmente à réplica 1 e *também* reencaminha a mensagem para o daemon 2 via *gossip* TCP sobre o porto 4002. O daemon 2 entrega às réplicas 2 e 3.

Onde estão as perdas: no *salto entre daemons*. O *gossip* é best-effort; com rajadas de ≈ 1100 mensagens/segundo, o canal TCP pode não ter largura de banda suficiente, e mensagens são descartadas. No nosso teste, perdeu-se o registo `1913` — único em 3000. Uma vez perdido, as réplicas do nó 5002 ficaram congeladas para sempre.

3.4 Testes e resultados

Cenário 1 nó — 3 execuções:

- Exec 1: líder 1200 ms, janela 1162 ms, 1000/1000/0.
- Exec 2: líder 921 ms, janela 917 ms, 1000/1000/0.
- Exec 3: líder 800 ms, janela 796 ms, 1000/1000/0.
- Médias: líder 974 ms, janela 958 ms, sem perdas.

Cenário 2 nós — 3 execuções:

- Exec 1: líder 1246 ms, todas 1000/1000/0.
- Exec 2: líder 923 ms, R1 1000/1000/0; R2 e R3 997/912/85.
- Exec 3: líder 849 ms, R1 1000/1000/0; R2 e R3 1000/0/1085.

A diferença entre *recebidos* e *escritos* explica-se pelo `seq`: *recebidos* conta todos os datagramas bem-formados; *escritos* só os que satisfazem `seq == last + 1`. Uma vez perdido o registo 1913, as réplicas do nó 5002 nunca mais escreveram.

4 Verificação — Ponto 14 (Falhas)

1. **Arrancar o líder e registar 3 registos antes de arrancar as réplicas**
   - Mecanismo: Multicast
   - Lado onde surgiu: nenhum
   - Exceção obtida: nenhuma
   - Ficheiros divergiram? Sim
   - Conclusão: o líder envia sem saber que ninguém estava à escuta — perda silenciosa. Falta de desacoplamento no tempo.
2. **Réplica com endereço de grupo** `230.0.0.2`
   - Mecanismo: Multicast
   - Lado: nenhum
   - Exceção: nenhuma
   - Divergiram? Sim
   - Conclusão: a rede só entrega a quem fez `joinGroup` no grupo exato para o qual o emissor enviou. Exclusão silenciosa.
3. **Buffer da réplica com 10 bytes**
   - Mecanismo: Multicast
   - Lado: réplica
   - Exceção: `IllegalArgumentException` (apanhada internamente)
   - Divergiram? Sim
   - Conclusão: o `receive()` trunca o datagrama; o `fromLine` deteta o formato inválido e imprime `REJEITADO`. A réplica continua à escuta — sem a proteção do ponto 6, terminaria.
4. **Parar a réplica a meio de** `rajada 1000`
   - Mecanismo: Multicast
   - Lado: nenhum
   - Exceção: nenhuma
   - Divergiram? Sim
   - Conclusão: a réplica perdeu mensagens e fica em `EM ESPERA` permanentemente. Sem recuperação.
5. **Daemon IPFS do líder parado**
   - Mecanismo: PubSub
   - Lado: líder
   - Exceção: `ConnectException` (subclasse de `IOException`)
   - Divergiram? Sim
   - Conclusão: ao contrário das falhas multicast, esta **produz exceção explícita** no lado do líder. O HTTP sabe que o daemon não responde.
6. `Pubsub.Enabled=false` **no nó das réplicas**

- Mecanismo: PubSub
- Lado: réplica
- Exceção: `IOException` (HTTP 500)
- Divergiram? Sim
- Conclusão: o daemon recusa o pedido de subscrição; a réplica não chega a subscrever e termina ao arrancar.

7. **Réplica a subscrever o tópico `rep01-temp`**

   - Mecanismo: PubSub
   - Lado: nenhum
   - Exceção: nenhuma
   - Divergiram? Sim
   - Conclusão: tópico errado = exclusão silenciosa. O Kubo aceita subscrever qualquer tópico, mesmo sem publicadores.

8. **Dois nós sem ligação entre si**

   - Mecanismo: PubSub
   - Lado: nenhum
   - Exceção: nenhuma
   - Divergiram? Sim
   - Conclusão: sem `swarm connect`, o *gossip* não funciona. Cada nó forma uma ilha PubSub isolada.

5 Verificação — Ponto 15 (Construções)

1. `MulticastSocket` / `joinGroup()`

   - Onde: `MulticastReplica`
   - Garantido: a réplica passa a receber datagramas entregues pela rede nesse grupo; a rede faz a replicação sem o emissor conhecer quem são.
   - Não garantido: entrega, ordem, ausência de duplicados, conhecimento do emissor sobre quem está subscrito.

2. `setTimeToLive(1)`

   - Onde: `MulticastLeader`
   - Garantido: o datagrama não sai da rede local (1 router máximo). TTL=0 limita à própria máquina; TTL=32 permitiria atravessar 32 routers, mas a maioria não encaminha multicast.
   - Não garantido: entrega, ordem, ausência de duplicados.

3. `append` local antes da propagação

   - Onde: `MulticastLeader` e `PubSubLeader`
   - Garantido: o líder nunca propaga um registo que não esteja já no seu ficheiro. Consistência da fonte autoritária.
   - Não garantido: se o líder falhar entre `append` e `send`, o líder fica à frente das réplicas.

4. Número de sequência (`seq`)
    - Onde: todas as classes
    - Garantido: detetar duplicados, lacunas e desordem; ficheiro sempre escrito pela ordem do líder.
    - Não garantido: recuperar registos perdidos. O `seq` deteta, não corrige.
5. Representação textual (`toLine`/`fromLine`)
    - Onde: `SensorRecord`
    - Garantido: interoperabilidade entre linguagens; formato explícito `seq;sensor;temperatura;instante`.
    - Não garantido: validação semântica (intervalo, formato de timestamp, plausibilidade do sensor).
6. Tópico `rep01-temperaturas`
    - Onde: `PubSubLeader`, `PubSubReplica`
    - Garantido: desacoplamento no espaço — o emissor publica num nome, não num endereço.
    - Não garantido: subscritores existentes, armazenamento (não guarda nada), entrega, ordem.
7. Daemon IPFS e API HTTP
    - Onde: `IpfsPubSub`
    - Garantido: comunicação com o nó local via HTTP.
    - Não garantido: daemon vivo, PubSub ativado, propagação para outros nós.
8. Codificação multibase
    - Onde: `IpfsPubSub`
    - Garantido: tópico e dados passam como texto seguro em HTTP (base64url sem padding, prefixo `u`).
    - Não garantido: sem multibase, o Kubo recusa o pedido ou interpreta mal a string.

6 Comparação e Reflexão

6.1 Ponto 16 — Tabela comparativa

Multicast:

- Tempo de envio do líder: não medido com rigor (contaminado por estado anterior).
- Latência da rajada até à réplica mais lenta: ≈ 45 ms.
- Débito no líder: elevado (rajada em ≈ 45 ms).
- Registos perdidos por réplica: variável; até $\approx 20\%$ numa execução limpa; 0% noutra.
- Duplicados e retidos no fim: `pending` a crescer sem limite se houver bloqueio.
- Componentes além da JVM: nenhum.
- Passos de configuração até ao primeiro registo: zero.

- O que aconteceu à réplica atrasada: ficou congelada em `EM ESPERA`.
- Funcionaria entre redes sem alterar routers? Não (a maioria não encaminha multicast).

PubSub 1 nó:

- Tempo de envio do líder: ≈ 974 ms (média de 3).
- Janela de receção: ≈ 958 ms.
- Débito no líder: ≈ 1030 reg/s.
- Registos perdidos: 0.
- Duplicados e retidos: 0.
- Componentes além da JVM: 1 daemon IPFS (porto 5001).
- Configuração: `ipfs init`, `ipfs config Pubsub.Enabled true`, `ipfs daemon`.
- O que aconteceu à réplica atrasada: ficou congelada em `EM ESPERA`.
- Funcionaria entre redes? Sim, desde que os nós se liguem via *gossip*.

PubSub 2 nós:

- Tempo de envio do líder: ≈ 1006 ms.
- Latência até à réplica mais lenta: ≈ 1006 ms.
- R1 (nó 5001): 1000/1000/0.
- R2 e R3 (nó 5002): perdas na execução 2 (997/912/85) e congelamento na execução 3 (1000/0/1085).
- Componentes além da JVM: 2 daemons + `swarm connect`.
- Configuração: segundo repositório, portas distintas, `swarm connect`.

Diferença *recebidos* vs *escritos*: *recebidos* conta datagramas bem-formados; *escritos* só os que satisfazem `seq == last + 1`. Como o registo 1913 se perdeu no *gossip*, as réplicas do nó 5002 ficaram bloqueadas em `EM ESPERA` e nunca mais escreveram — acumularam tudo em `pending`. Nas réplicas do nó 5001, que receberam 1913 sem perdas, *recebidos* e *escritos* coincidem em 1000. Isto ilustra o que o `seq` permite detetar (há um salto) e o que *não* permite resolver (recuperar o registo em falta).

6.2 Ponto 17 — Reflexão

Desempenho. O multicast foi significativamente mais rápido (janela ≈ 45 ms) do que o PubSub ($\approx$ 950–1000 ms), sendo o PubSub cerca de 20 vezes mais lento. No multicast, o tempo gasta-se entre `send` e `receive` (praticamente instantâneo na LAN). No PubSub, há overhead de HTTP POST para o daemon, do próprio daemon, do *gossip* e do stream HTTP de subscrição.

Escalabilidade. No multicast, o custo de envio do líder é constante (1 datagrama para 1 ou 100 membros). No PubSub, cada nó reencaminha para alguns vizinhos, distribuindo o tráfego pela malha — o tráfego total cresce com o número de nós, mas distribui-se. Com 300 réplicas na Internet, o multicast deixa de funcionar (routers não encaminham) e só o PubSub escala.

Fiabilidade. O PubSub de 1 nó não perdeu nada. O multicast perdeu $\approx 20\%$ numa rajada limpa. O PubSub de 2 nós perdeu o registo 1913 no *salto entre daemons*. O `seq` permitiu *detetar* o salto, mas não *recuperar* — nenhuma réplica pede o registo em falta.

Recuperação. Faltaria um mecanismo de retransmissão pedido pela réplica ao líder, ou transferência de estado — ambos fora do âmbito desta ficha. Com os mecanismos atuais, uma réplica atrasada fica congelada para sempre.

Dependências e alcance. Para sensores num só edifício, o multicast é mais simples (só JVM) e mais rápido. Para sensores em várias cidades, só o PubSub funciona, embora exija um daemon Kubo por máquina.

Ponto único de falha. Se o líder falhar, deixa de haver escritas nos dois mecanismos — nenhuma das soluções resolve isto. Resolver exigiria eleição de líder ou consenso, ambos fora do âmbito.

Segurança. Qualquer processo na rede pode enviar datagramas para `230.0.0.1:6789` ou publicar em `rep01-temperaturas`. As réplicas não distinguem o líder de um atacante (não há autenticação). O campo `from` identifica o PeerID do nó que publicou, mas não é verificado — uma validação exigiria que a réplica conhecesse à partida o PeerID do líder e o comparasse antes de processar. Está fora do âmbito desta ficha.

7 Conclusão

Nesta ficha foram implementados e comparados dois mecanismos de comunicação indireta aplicados à replicação de um ficheiro de registos com líder único.

O **multicast** é simples e rápido na rede local, mas não tem desacoplamento no tempo nem deteção de perdas. O **PubSub do IPFS** atravessa a Internet via *gossip* e oferece desacoplamento no espaço, mas exige um daemon por máquina e continua best-effort, com perdas possíveis no *salto entre nós*.

Em ambos, o `seq` *deteta* lacunas, mas não as *recupera*. Falhas que produzem exceção (`ConnectException`, HTTP 500) coexistem com falhas silenciosas (grupo errado, tópico diferente, nós sem ligação). A distinção entre falhas *detetáveis* e falhas *silenciosas* é a conclusão central: **enviei não é o mesmo que recebeu**, e nem o multicast nem o PubSub garantem essa distinção por si só.